

SG EduCoach

Mevcut sistem ile HTML, CSS, JavaScript ve C# tabanlı yeni sistemin karşılaştırması

SONUÇ

Sistem C# ile baştan sona yeniden geliştirilebilir. Kullanıcı deneyimi ve temel işlevler korunabilir; en büyük değişiklik arka uç mimarisi, kimlik doğrulama, güvenlik ve yayınlama yönteminde olur.

Temel teknoloji farkları

Konu	Şu anki sistem	C# ile yeni sistem
Arayüz	Next.js, React ve TypeScript	HTML5, CSS3 ve Vanilla JavaScript
Sunucu	Next.js sunucu işlemleri	ASP.NET Core C#
Veritabanı	Supabase PostgreSQL	PostgreSQL ve Entity Framework Core
Kullanıcı girişi	Supabase Auth	ASP.NET Core Identity
Yetkilendirme	Supabase RLS ve uygulama kontrolleri	C# rol ve politika kontrolleri
Yayınlama	Vercel ve Supabase	IIS, Linux, Docker veya Azure
Anlık güncellemeler	React durum yönetimi	Fetch/AJAX veya SignalR
Arka plan işleri	Vercel Cron	Hosted Services veya Hangfire
Grafikler	Recharts	Chart.js veya benzeri JavaScript kütüphanesi

Kullanıcının görebileceği değişiklikler

- Sayfa adresleri ve genel tasarım istenirse mevcut sistemle aynı tutulabilir.
- Bazı sayfalarda tam sayfa yenilemesi görülebilir; AJAX kullanımıyla bu etki azaltılabilir.
- Formlar, grafikler, mobil menüler ve tema sistemi yeniden uygulanacağı için küçük görsel farklılıklar oluşabilir.
- PWA kurulumu, çevrimdışı ekranı ve Web Push bildirimleri korunabilir.
- SignalR kullanılırsa bazı bildirimler gerçek zamanlı çalışabilir.
- İyi optimize edilmiş bir uygulama mevcut sistemle aynı veya daha iyi performans verebilir; teknoloji değişimi tek başına hız garantisi değildir.

Yönetim ve teknik taraftaki farklar

Daha merkezî yönetim

Kullanıcı, rol, izin, loglama ve hata takibi C# uygulamasında tek merkezden yönetilebilir.

Daha fazla barındırma seçeneği

Uygulama kurum içi IIS sunucusunda, Linux üzerinde, Docker ile veya Azure'da çalıştırılabilir.

Bakım sorumluluğu

Sunucu güvenliği, güncellemeler, yedekleme ve izleme yeni altyapının işletmecisine ait olur.

Veritabanı yönetimi

Şema değişiklikleri Entity Framework Core migration'larıyla takip edilebilir.

En önemli risk

En büyük iş yükü ekranları çizmek değil; mevcut veritabanı kurallarını, rol sistemini ve güvenlik kontrollerini eksiksiz taşımaktır. Supabase Row Level Security kurallarının karşılığı C# tarafında doğru kurulmazsa veri erişim açıkları oluşabilir.

- Öğrenci-veli eşleştirmeleri ve okul bazlı kullanıcı numaraları
- Öğretmen, müdür, moderatör ve yönetici yetkileri
- Geçici şifreler, hesap aktifliği ve oturum güvenliği
- Deneme sonuçları, rozetler, duyurular ve hatırlatmalar
- Mevcut kullanıcı ve operasyon verilerinin kayıpsız aktarılması

Önerilen C# mimarisi

Katman	Öneri
Arayüz	Razor görünümleri + HTML + CSS + gerektiği yerlerde Vanilla JavaScript
Uygulama	ASP.NET Core MVC
Veri	PostgreSQL + Entity Framework Core
Güvenlik	ASP.NET Core Identity + rol/politika tabanlı yetkilendirme
Gerçek zaman	SignalR
Zamanlanmış işler	Hosted Services veya Hangfire
Dış servisler	Claude API, Resend/SMTP ve Web Push

Uygulama yaklaşımı

1. Yeni C# çözümünü mevcut çalışan sistemden ayrı kurmak.
2. Veritabanı modelleri, giriş sistemi ve rol yetkileriyle başlamak.
3. Öğrenci, öğretmen, veli ve yönetim ekranlarını modül modül taşımak.
4. Her modülü mevcut sistemle karşılaştırmalı olarak test etmek.
5. Veri aktarımı ve güvenlik kontrolleri tamamlandıktan sonra canlı geçiş yapmak.

Genel değerlendirme

İşlevlerin tamamı korunabilir ve sistem C# merkezli, kurumsal sunucu ortamlarına daha uygun bir yapıya dönüştürülebilir. Ancak bu çalışma basit bir kod çevirisi değil; güvenlik, veri ve kullanıcı deneyimiyle birlikte yürütülmesi gereken kapsamlı bir yeniden geliştirme projesidir.